APE: Dynamic Workload Configuration Syntax

Member

- Shiva Jahangiri
- Hongyu Shi

Motivation

As APE8: Dynamic Workload Management proposed, users can define query groups along with their respective priority levels.

To allow users to have such configuration in a schematic way, we propose storing scheduler configurations in a dedicated metadata table. This allows users to easily add or remove query groups, adjust their priorities, and fine-tune parameters related to the dynamic workload module using the new syntax outlined in this proposal.

Proposed changes

Priority levels are represented as integers starting from 1, where 1 indicates the lowest priority. AsterixDB will store metadata about these query groups and their associated priorities. For more details on the scheduler's architecture, please refer to the link to the previous proposal.

To clarify the proposed syntax, below is an example of a list of query groups and their corresponding priority levels for an initial configuration.

```
ui: 4
analytics: 6
management: 8
```

1. DDL

Create a scheduler configuration for a customized workload:

`CREATE SCHEDULER CONFIG` `${configuration_name}` `(IF NOT EXISTS AS)` {...};

Use the keyword `DROP` to remove an existed configuration:

`DROP SCHEDULER CONFIG` `${configuration_name}` `(IF EXISTS)` ;

```
CREATE SCHEDULER CONFIG s_config_1 AS{
   "defaultPriority": 1,
   "shortMemoryPercent": 10,
   "shortCPUQuota": 20
   "queryGroup":
   [
      {
         "priority": 4,
         "grouplist": [ "ui"]
      },

      {
         "priority" : 6,
         "grouplist": ["analytics", "adhoc"]
      },

      {
         "priority": 8,
         "grouplist": [ "management" ]
      }
   ]
};
```

```
DROP SCHEDULER CONFIG s_config_0 IF EXISTS;
```

Parameters:

- `defaultPriority` [POSITIVE INTEGER]: Priority of non-tagged query.
- `shortMemoryPercent` [POSITIVE FLOAT]: Percent of cluster's memory user wants to assign to short-tagged query.

- `shortCPUQuota` [POSITIVE FLOAT]: Percent of cluster's CPU quota user wants to assign to short-tagged query.
- `queryGroup` [UNORDERD LIST OF OBJECTS]: Each item in the list has fields `priority` [POSITIVE INTEGER] and `grouplist` [UNORDERED LIST]; `priority` indicates the priority assigned to all groups in `grouplist` , listing groups' names.
    - Each group's name has to contains two characters at least. e.g. **ui** and **analytics** are valid group names, while **a** is not.

Users can define multiple scheduling configurations. To activate `s_config_1` as shown above, a new keyword `ENABLE` is introduced, allowing a specific configuration to take effect.
The configuration applies cluster-wide rather than being limited to a specific dataverse.

```
ENABLE SCHEDULER CONFIG s_config_1
```

Each user's workload configuration stored in dedicated metadata dataset as a record of the type `WorkloadConfigType`. The record type description is shown in the 5.Appendix section.

2. Query Scheduling

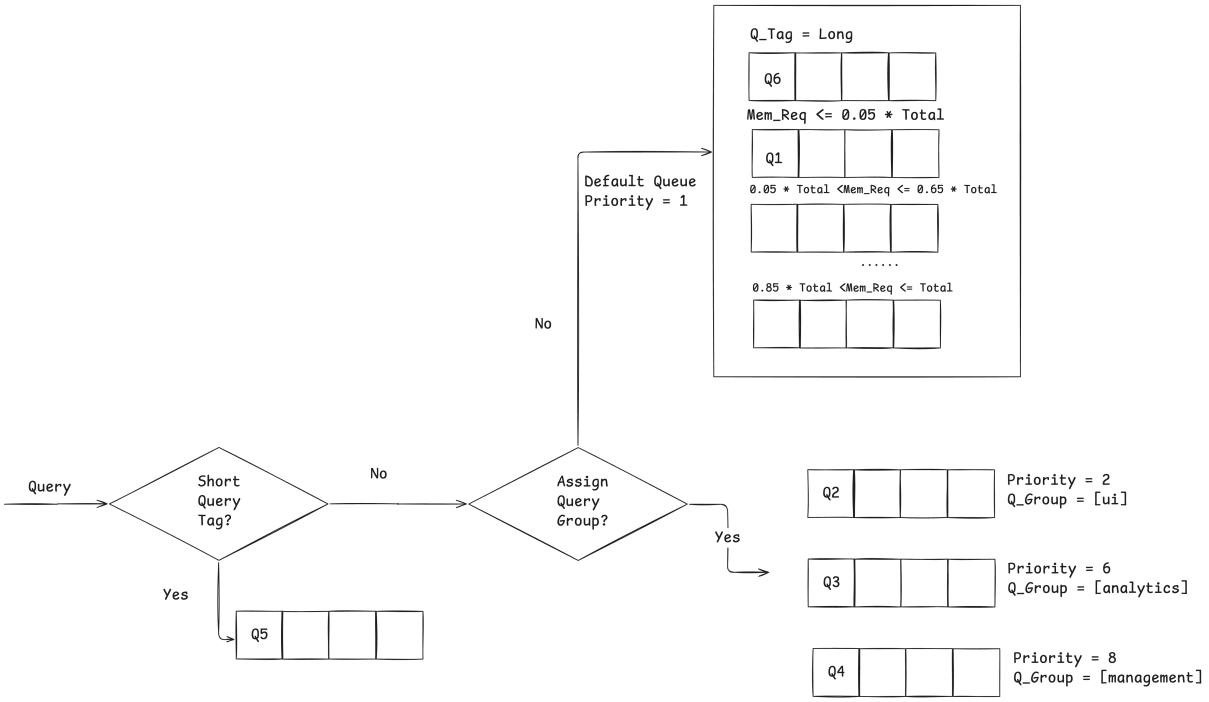
Once `s_config_1` is enabled, users can assign queries to the newly defined query groups. The keyword `SET` is used to specify the name of the group preceding each query statement.

The following six queries are submitted to the system simultaneously. *${...} represents the query statement and the specific contents is not significant here.*

```
${Query1 Body ...}

/* Indicate group in the query */
SET 'qgroup' `ui`;
${Query2 Body ...}

SET 'qgroup' `analytics`;
```

```
${Query3 Body ...}

SET 'qgroup' `management`;
${Query4 Body ...}

/* Short-tagged query */
SET 'qgroup' `s`;
${Query5 Body ...}

/* Long-tagged query */
SET 'qgroup' `l`;
${Query6 Body ...}
```

Based on our policy, six queries will be scheduled to different queues, as graph below (assume query1's memory requirement is low):



3. DML

3.1 Upsert/Delete Groups

Upsertion

`UPSERT QGROUP INTO` `${configuration_name}` {[...]};

```
UPSERT QGROUP INTO s_config_1 {[
      {
         "priority": 10,
         "name": "ingest"
      },

      {
         "priority": 6,
         "name": "management"
      }
]}
```

Each upsertion is represented as a priority-name pair object within a list. The statement below inserts a new group, `ingest`, with a priority of 8 and updates the priority of `management` from 8 to 6. After applying the upsertion statement on `s_config_1`, the `queryGroup` field will appear as follows:

```
   "queryGroup":
   [
      {
         "priority": 4,
         "grouplist": [ "ui"]
      },

      {
         "priority" : 6,
         "grouplist": ["analytics", "management"]

      },

      {
         "priority": 10,
         "grouplist": [ "ingest" ]
      }
   ]
```

Deletion

`DELETE QGROUP FROM` `${configuration_name}` {[...]};

Groups to be deleted are indicated as a list of strings, which include names to be deleted.

```
DELETE GROUP FROM s_config_1 {[
   "analytics", "ingest"
]}
```

After the deletion statement on `s_config_1` , its `queryGroup` field will look like:

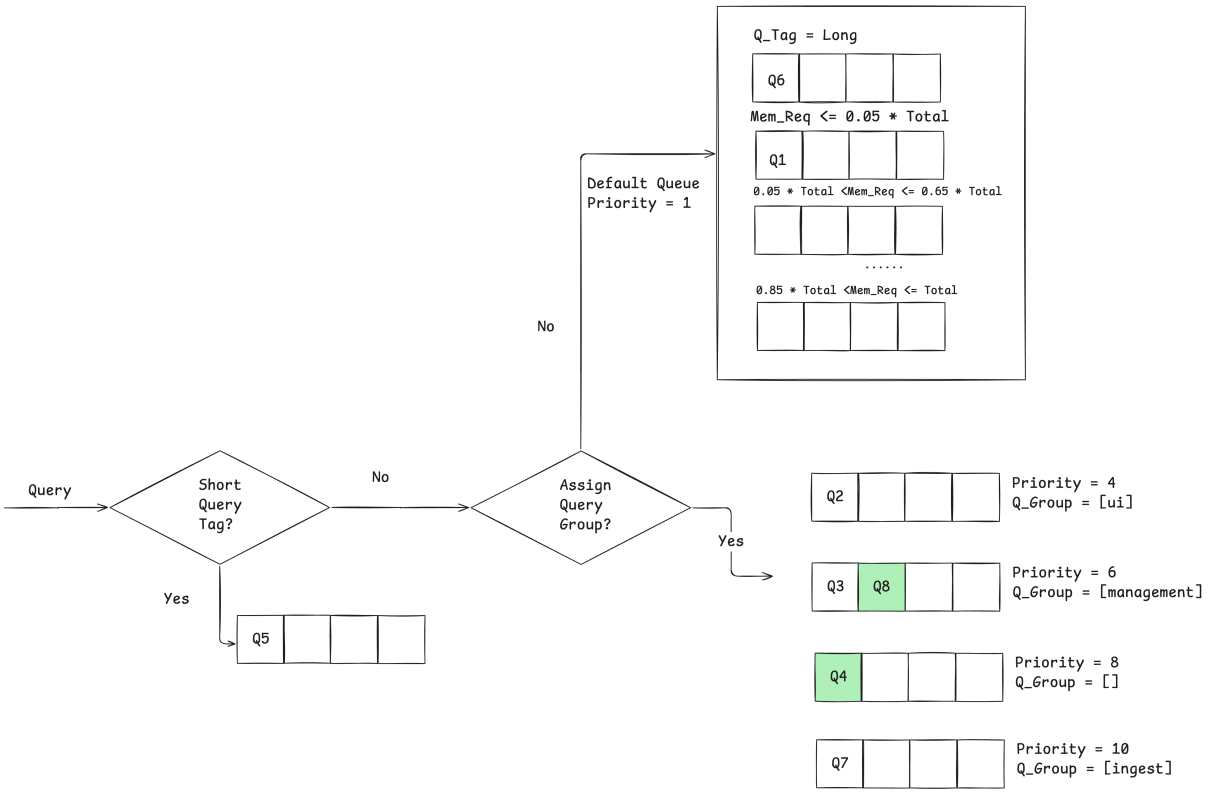
```
   "queryGroup":
   [
      {
         "priority": 4,
         "grouplist": ["ui"]
      },

      {
         "priority" : 6,
         "grouplist": ["management"]

      },

      {
         "priority": 10,
         "grouplist": [ "ingest" ]
      }
   ]
```

Now, we have three more queries below sent to the system at the same time.

```
SET 'qgroup' `ingest`;
${Query7 Body ...}

/* Indicate group in the query */
SET 'qgroup' `management`;
${Query8 Body ...}
```

```
SET 'qgroup' `analytics`;
${Query9 Body ...}
```

If query1-query6 haven't been processed by the system, they will be scheduled as graph below.



- At this time, Q7 is scheduled to the new queue with priority of 10.
- Green block indicates query set as `management` group. Since `management` 's priority has been modified to 6, Q8 is scheduled to the queue with priority of 6.
- Group `analytics` has been deleted, so there are no matched queue for Q9, which causes an exception alerting that `analytics` does not exists.

3.2 Change Other Workload Parameters.

To update other parameters in configuration, use syntax:

`UPDATE SCHEDULER CONFIG` `${configuration_name}` {};

The example below modifies `defaultPriority` to 2, so queries waiting in the *Default Queue* will have higher chance to be admitted by the scheduler.

```
UPDATE SCHEDULER CONFIG s_config_1 {
   "defaultPriority": 2,
   "shortMemoryPercent": 15,
   "shortCPUQuota": 15
}
```

3.3 List Configuration Details

To list all existed scheduler configurations:

`SELECT VALUE config FROM Metadata.`SchedulerConfig` config;`

To check currently enabled configuration's information:

`SELECT VALUE state FROM Metadata.`SchedulerState` state;`

If `s_config_1` is currently enabled, the statement dumps all details about the particular configuration.

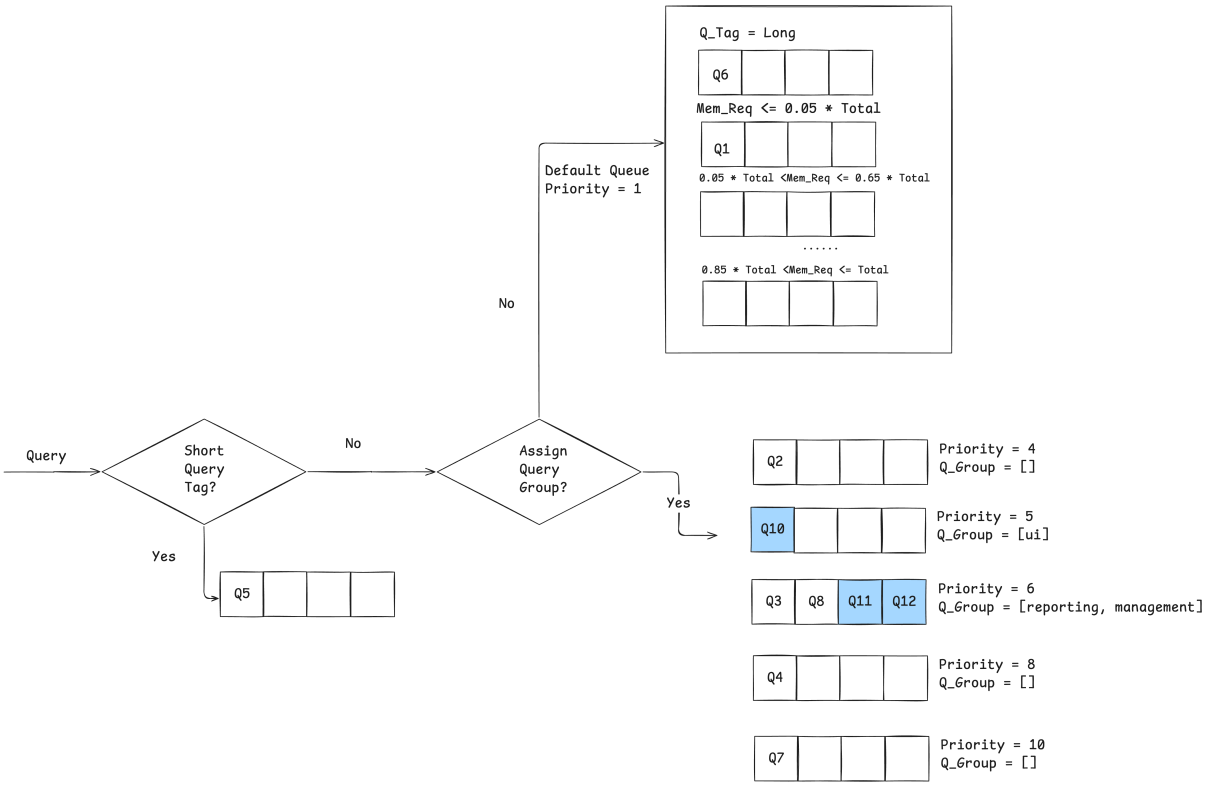
4. Switch Between Configurations

User can switch between different configurations for scheduling. In example below, user create a separate `s_config_2` where it also has a group `ui` with different priority from `s_config_1` , a group `management` with the exactly same priority, and a new group `reporting`. This change only affects new queries sent after enabling `s_config_2`. Queries submitted before this switch will remain in their respective queues with the previous priorities.

```
CREATE WORKLOAD CONFIG s_config_2 {
   "defaultPriority": 3,
   "shortMemoryPercent": 10,
   "shortCPUQuota": 20
   "queryGroup":
   [
      {
         "priority": 5,
         "grouplist": [ "ui"]
      },

      {
```

```
        "priority" : 6,
        "grouplist": ["reporting", "management"]

      }
   ]
};

ENABLE SCHEDULER s_config_2

SET 'qgroup' `ui`;
${Query10 Body ...}

/* Indicate group in the query */
SET 'qgroup' `reporting`;
${Query11 Body ...}

SET 'qgroup' `management`;
${Query12 Body ...}
```

We have three more queries sent to the system at the same time. Still, we assume that query1-query8 haven’t been processed yet. They will be scheduled as graph below.



Q10 - Q12, highlighted in blue, are newly scheduled to their respective queues. Once Q2, Q4, and Q7 have been admitted for execution, the queues with no associated group will be removed.

5. Appendix

- Each user's workload configuration stored in metadata as a record of the type `WorkloadConfigType`. The description is as follows.

```
{ "DataverseName": "Metadata",
   "DatatypeName": "SchedulingConfigType",
   "Derived": {"Tag": "RECORD",
           "IsAnonymous": false,
           "Record": {
            "IsOpen": true,
            "Fields": [
             {
              "FieldName": "SchedulingConfigName",
                          "FieldType": "string",
              "IsMissable": false
```

```
            },
            {
              "FieldName": "DefaultPriority",
              "FieldType": "string",
              "IsMissable": true
            },
            {
              "FieldName": "ShortMemoryPercent",
              "FieldType": "string",
              "IsMissable": true
            },
            {
              "FieldName": "ShortMemoryQuota",
              "FieldType": "string",
              "IsMissable": true
            },
            {
              "FieldName": "QueryGroups",
              "FieldType": "ORDEREDLIST",
              "IsMissable": true
            },
          ]
        }
       },
  }
```

- APE: Dynamic Workload Management
- Colorado-V2 scheduling
- Reference to syntax of full-text configurations and filters
- Configure Query Priority in Amazon Redshift WLM

Compatibility / Migration

The proposed changes do not modify any existing flow or behavior and can be seen strictly as an extension to the system's global metadata features.